

Introduction

You've probably felt this before: the code isn't doing anything unusual, the logic checks out, and yet it crawls. Nothing looks broken, but it's slow — and that gap between "looks fine" and "runs fine" is exactly where a lot of real-world Python performance problems live.

This matters for two reasons. First, in interviews, correctness alone doesn't always cut it — interviewers frequently probe *why* your solution is efficient, not just whether it produces the right answer. Second, in production code, these habits compound. A pattern that costs a few milliseconds on a 10-item list can cost minutes on a 10-million-item dataset.

The good news is that almost none of this requires exotic knowledge. It's a short list of habits — how you build strings, which data structure you reach for, whether you're recalculating things you don't need to — that are easy to fix once you know to look for them. Let's go through all ten, build the intuition behind each one, and end with the meta-skill that actually matters more than memorizing the list: measuring instead of guessing.

Problem Overview

The underlying question is simple: **given a piece of working Python code, why might it run far slower than it needs to, even though the logic is correct?**

This isn't one bug — it's a set of independent patterns, each of which trades away performance for what looks like convenience or habit. They fall into a few buckets:

- **Doing more copying than necessary** (strings, lists)
- **Doing more searching than necessary** (checking membership in the wrong structure)
- **Doing more computing than necessary** (repeated work, unnecessary object creation)
- **Doing more lookup work than necessary** (globals, dotted attribute access)
- **Not using faster machinery that already exists** (built-ins, the right data structure)
- **Not measuring at all** (optimizing blind)

Understanding each one individually is useful. Understanding the *shape* they share — unnecessary work repeated many times — is what lets you spot new instances you've never seen before.

Example

Say you're building a report by joining log lines into one string, and checking each new line against a list of "already seen" entries to avoid duplicates:

```
seen = []
report = ""
for line in logs:
    if line not in seen:          # membership check
        report += line + "\n"    # string concatenation
        seen.append(line)
```

For 100 log lines, this finishes instantly. For 500,000 log lines, this can take minutes — not because the machine is slow, but because both the `in seen` check and the `+=` are doing far more work per iteration than they need to. That gap between "small input, fine" and "large input, unusable" is the tell that you've hit one of the ten patterns below.

Intuition

Before touching code, an experienced engineer asks one question repeatedly: **"How much total work does this do as the input grows, and is any of that work avoidable?"**

Two follow-up instincts come out of that:

1. **Is something being rebuilt that could just be extended?** Strings and lists that grow via `+=` inside a loop often get fully recopied each time, rather than appended to.
2. **Is something being searched that could be looked up instead?** Checking "is X in this collection" is fundamentally a search problem, and different data structures solve search at wildly different speeds.

Once you internalize those two questions, most of the ten habits collapse into two families: **needless copying** and **needless searching/recomputing**. The rest — globals, built-ins, wrong data structures — are specific instances of those same ideas.

Brute Force Solution

The "brute force" version of almost any of these problems is the natural first draft: write a loop, do the obvious operation each iteration, don't think about the underlying data structure's complexity.

Idea: Solve the problem the most direct, literal way — loop through data, mutate a string or list with `+=`, check membership with `in` on a list, recompute anything you need fresh each time.

Algorithm: 1. Initialize an empty accumulator (string, list, or counter). 2. For each item, perform the operation directly against that accumulator. 3. Repeat until done.

Advantages: - Obvious to write and to read. - Correct — it produces the right answer. - Fine for small inputs (tens to low hundreds of items).

Disadvantages: - Each operation may silently cost more than it looks like it does (e.g., `+=` on a string is $O(n)$, not $O(1)$). - Costs compound across iterations, turning an $O(n)$ *intent* into $O(n^2)$ *actual* behavior. - Doesn't scale — the same code that was instant in testing becomes unusable in production.

Time complexity: Often $O(n^2)$ in practice, even though the code reads like $O(n)$. **Space complexity:** $O(n)$, but with far more intermediate allocation than necessary (each recopy is a temporary object).

Optimal Solution

The optimal version isn't a single trick — it's applying the right primitive to each sub-problem. Here's the mapping:

Symptom	Root Cause	Fix
<code>result += item</code> in a loop (string)	Strings are immutable; each <code>+=</code> copies everything before it once	Collect into a list, <code>"".join(list)</code>
<code>result += [item]</code> in a loop (list)	Same copying problem, different container	Use <code>.append()</code> / <code>.extend()</code>
Slow <code>item</code> in collection	Linear scan on a list	Use a <code>set</code> or <code>dict</code> (hash-based $O(1)$ lookup)
Recomputing the same value every iteration	Work that doesn't depend on the loop variable is redone anyway	Hoist it out of the loop, compute once
Slow attribute/variable access	Global and <code>self.</code> lookups traverse more scope than a local variable	Cache the value in a local variable before the loop
Hand-written loop doing built-in work	Every step pays Python's interpreter overhead	Use <code>map</code> , comprehensions, <code>sum</code> , <code>sorted</code> , or NumPy — implemented in C
Wrong container for the access pattern	List = linear scan, dict/set = hash jump	Pick dict/set when you need frequent lookups
Repeated deep copies inside a loop	Every copy walks and rebuilds the whole structure	Build read-only data once, outside the loop

The shared principle underneath all eight rows: **move fixed-cost work outside the loop, and match the data structure to the operation you perform most.**

Python Code

```
def build_report(logs: list[str]) -> str:
    """Join unique log lines into a single report string efficiently."""
    seen: set[str] = set()      # O(1) membership checks
    unique_lines: list[str] = [] # accumulate, don't concatenate

    for line in logs:
        if line not in seen:    # hash lookup, not linear scan
            seen.add(line)
            unique_lines.append(line) # append, not +=

    return "\n".join(unique_lines) # single join, not repeated copies
```

Code Walkthrough

- `seen: set[str] = set()` — a set gives O(1) average-case membership checks, replacing the O(n) scan a list would require.
- `unique_lines: list[str] = []` — a plain list used with `.append()`, avoiding any string recopying during the loop.
- `if line not in seen` — this is the fast membership check; on a list this line alone would turn the whole function into O(n²).
- `seen.add(line)` — O(1) insertion, keeping future lookups fast.
- `unique_lines.append(line)` — O(1) amortized append, no copying of prior elements.
- `"\n".join(unique_lines)` — builds the final string exactly once, instead of rebuilding it on every iteration like `+=` would.

Dry Run

Input: `logs = ["error A", "error B", "error A", "error C"]`

Step	line	in seen?	seen	unique_lines
1	"error A"	No	{"error A"}	["error A"]
2	"error B"	No	{"error A", "error B"}	["error A", "error B"]
3	"error A"	Yes	(unchanged)	(unchanged)
4	"error C"	No	{"error A", "error B", "error C"}	["error A", "error B", "error C"]

Final `"\n".join(...)` produces:

```
error A
error B
```

error C

Complexity Analysis

- **Time:** $O(n)$ — each log line is processed once, and both the set lookup/insert and list append are $O(1)$ on average. The brute-force version with a list-based `seen` and string `+=` is effectively $O(n^2)$: $O(n)$ for the membership scan times $O(n)$ iterations, plus $O(n)$ per string copy.
- **Space:** $O(n)$ — one set and one list, each holding at most n elements, plus the final joined string. This is unavoidable since the output must contain all unique lines.
- **Why this is correct:** hash-based sets guarantee average $O(1)$ lookup/insert because they compute a hash once and jump to a bucket, rather than comparing against every existing element.

Alternative Solutions

If you don't need to preserve insertion order and just want unique lines with no report string, `set(logs)` alone does the deduplication in one line, at the cost of losing the original ordering (fixed in Python 3.7+ dicts, but not in plain sets). If memory is tighter than CPU time, a generator-based approach with `dict.fromkeys(logs)` (which does preserve order) can dedupe without materializing an intermediate set and list separately:

```
"\n".join(dict.fromkeys(logs))
```

This trades a slightly less obvious idiom for one fewer explicit container.

Edge Cases

- **Empty input** (`logs = []`) — the loop never executes; `join` on an empty list returns `""`, which is correct.
- **All duplicates** (`["a", "a", "a"]`) — `seen` prevents any duplicate from reaching `unique_lines`; output is `"a"`.
- **Single element** — works identically to the general case, just with one iteration.
- **Very large input** (millions of lines) — this is exactly where the $O(n)$ approach separates from the $O(n^2)$ brute-force version in real, measurable time.
- **Non-string elements** — if `logs` contains non-hashable items (like lists), `set` membership checks will raise a `TypeError`; the function assumes hashable, string-like input.

Common Mistakes

1. Using `+=` to build a string inside a loop instead of collecting into a list and joining once.
2. Using `+=` to grow a list instead of `.append()` / `.extend()`.
3. Checking membership against a `list` when a `set` or `dict` would do the same job in constant time.
4. Recomputing a value inside a loop that never changes between iterations.
5. Relying on global variables or repeated `self.attr` lookups instead of caching into a local variable.
6. Writing a manual loop for something a built-in (`sum`, `sorted`, `map`, list comprehension, NumPy) already does faster in C.
7. Deep-copying data inside a loop when the data is never mutated and could be built once outside it.
8. "Optimizing" based on a guess about where the code is slow, instead of profiling first.

Interview Questions

- "Why is `list` membership checking $O(n)$ but `set` / `dict` membership checking $O(1)$ on average?"
- "What's the actual time complexity of building a string with `+=` in a loop, and why?"
- "How would you profile a Python function to find its actual bottleneck?"
- "When would a list still be the better choice over a set or dict, despite slower membership checks?"
- "Why are built-in functions like `sum()` or `sorted()` typically faster than an equivalent hand-written loop?"

Similar Problems

- **Two Sum (LeetCode #1)** — the classic example of trading an $O(n^2)$ brute-force nested loop for an $O(n)$ solution using a hash map for lookups.
- **Contains Duplicate (LeetCode #217)** — directly tests whether you reach for a set instead of nested list comparisons.
- **Longest Substring Without Repeating Characters (LeetCode #3)** — relies on set/dict membership checks inside a sliding window for efficiency.
- **Group Anagrams (LeetCode #49)** — depends on choosing dict as the right structure for grouping instead of repeated list scans.

Key Takeaways

Slow Python code is rarely about "Python being slow" — it's almost always about doing more work per operation than the code appears to. Immutable strings punish `+=` in loops. Lists punish membership checks. Loops punish anything that could've been hoisted out or handed to a built-in. And no amount of intuition beats actually profiling before you optimize — measure first, fix the real bottleneck second.

Watch the Video

For the full walkthrough with a live loop-vs-built-in speed comparison, watch the video here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series built around making the fundamentals — algorithms, data structures, and the habits behind clean, efficient code — genuinely click, one problem at a time.

Call To Action

If this helped, drop a like on the video, subscribe on YouTube so you don't miss the next one, and subscribe to the Substack for the written breakdowns. Comments with your own "slow code" stories are always welcome — and if this saved you a debugging session, share it with someone who'd benefit.

Quick Review

You wrote a loop over 100k items and it's slow — first move?

Profile before guessing (cProfile/timeit); don't optimize blind, find which mistake actually applies.

String concat in a loop with `+=` — cost and fix?

$O(n^2)$ since each `+=` copies the whole string; use `".join(list)"` instead, $O(n)$.

Growing a list wrong — what's the antipattern?

Inserting at index 0 repeatedly ($O(n)$ each); `append()` is $O(1)$ amortized, or use `collections.deque` for left-pushes.

'x in my_list' inside a hot loop — why bad, what's the fix?

List membership is $O(n)$ scan; convert to a set/dict first for $O(1)$ average lookup.

Recomputing len(x) or a function call every loop iteration — fix?

Hoist invariant work outside the loop; compute once, reuse the cached value.

Global variable + dotted attribute lookup vs local var in a loop — why slower?

Global/attribute access re-resolves each iteration; local variable lookup is a fast array index, so alias to a local first.

Top 10 mistakes that slow down Python code — free Python cheat sheet